МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

ЛАБОРАТОРНАЯ РАБОТА № 1

«Введение в Apache Spark»

по курсу
Большие данные

Группа 6403
Студент_____ А.А. Кучумов
(*подпись*)

Преподаватель,
Ассистент каф. ТК _____ В.И. Проценко
(*подпись*)

Самара 2025

ЗАДАНИЕ

В первой части задания вы работаете с 2 файловыми системами:

1. файловой системой Linux узла кластера,

2. распределённой файловой системой.

Проведите анализ данных велопарковок на языке Python в интерактивном режиме из Jupyter книг:

1. L1_interactive_bike_analysis_python_with_rdd.ipynb,

2. L1_interactive_bike_analysis_python_with_dataframes.ipynb.

Проведите анализ данных велопарковок на языке Scala или Python в неинтерактивном режиме (--deploy-mode cluster). Инструкции по созданию и запуску приложений: Python – L1_noninteractive_bike_analysis_scala.py.

Решите задачи L1_Apache_Spark_Tasks.md.

ХОД ВЫПОЛНЕНИЯ

Задание на лабораторную работу 1 разделил на две условные части:

1. Работа с файловой системой Linux узла кластера (Spark, датасеты: «warandsociety», «nyctaxi»).

2. Работа по анализу данных велопарковок (интерактивный режим: RDD, Dataframes; неинтерактивный режим на сервере; решение задач).

Лабораторную работу я выполнял на операционной системе Windows и в качестве виртуальной машины для Linux-дистрибутива использовал WSL (Ubuntu 24.04). В репозитории с лабораторными работами был скрипт, с помощью которого удалось установить необходимые зависимости, а также настроить виртуальное окружение Python. Помимо прочего я установил spark-shell в систему, его корректная работа представлена на рисунке 1.

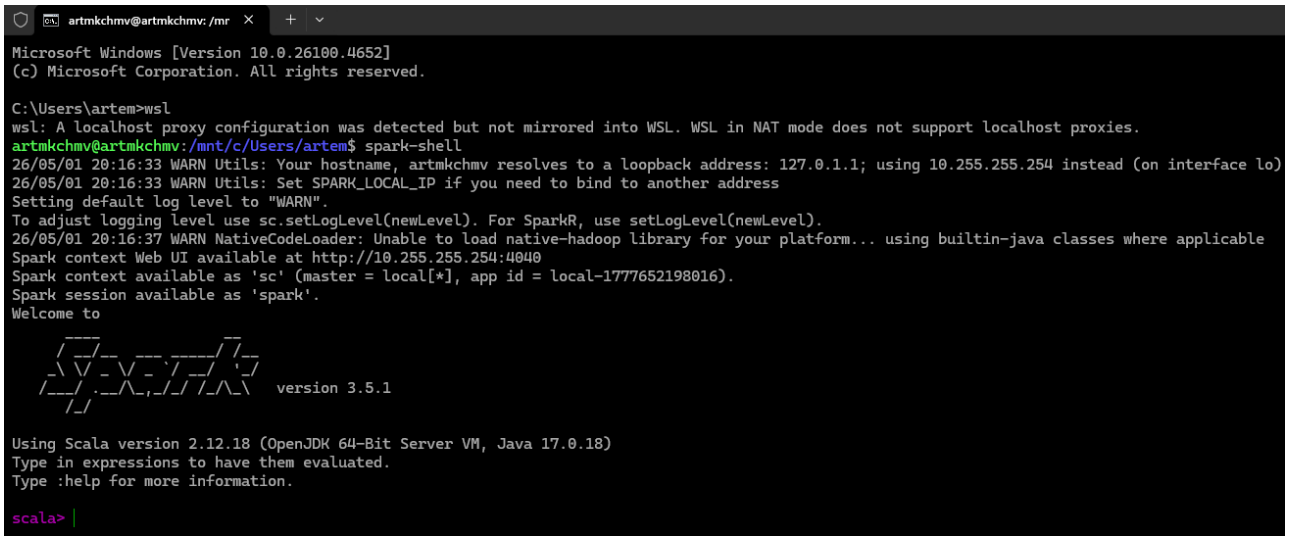

Рисунок 1 – Запущенная сессия spark-shell

Далее я приступил к выполнению первой части лабораторной работы. В первую очередь был считан файл «warandpeace.txt», который представлял из себя книгу «Война и мир». Я поработал с такими командами, как:

1. count() – возвращает общее количество элементов в RDD.

2. take(n) – возвращает список из первых n элементов.

3. getNumPartitions() – показывает, на сколько партиций разбит RDD.

4. parallelize() – создаёт RDD из локальной коллекции.

5. filter(func) – оставляет только элементы, удовлетворяющие условию.

6. first() – возвращает первый элемент RDD.

7. cache() – кэширует RDD в памяти для ускорения повторных вычислений.

После этого я поработал с операциями над множествами. Были созданы два списка с помощью parallelize() и проведены следующие операции:

1. union() – объединение.

2. distinct() – удаление дубликатов.

3. intersection() – пересечение.

4. subtract() – разность.

Также важно упомянуть, что после вышеупомянутых операций вызывалась команда collect(), которая собирает все элементы RDD со всех рабочих узлов.

Еще я попробовал поработать с широковещательными и аккумулирующими переменными. Первые создаются с помощью broadcast(); к их значению можно обратиться через поле value. Вторые создаются с помощью accumulator и используются, например, при параллельной обработке массива для добавления каждого обрабатываемого значения; к их значению также можно обратиться через поле value.

Далее необходимо было из набора данных о такси в городе Нью-Йорк получить 10 самых популярных номеров такси. Результат поиска представлен на рисунке 2.

```
[('"AB44AD9A03B7CFAF3925103BDCC0AF23"', 44),
 ('"71CACFBADF9568AAE88A843DB511D172"', 41),
 ('"6483B9BFCB216EC88986EA3AB13064E7"', 41),
 ('"4C73459B430339981D78795300433438"', 41),
 ('"67E71D24AF704D814A0A825005ADA72E"', 40),
 ('"02E5A4136FD0A775A023A005A4EABC62"', 40),
 ('"9DFBCD218E7116F34C044F0680A0FB8A"', 39),
 ('"8DEB70907D00AA1D7FF5E2683240549B"', 39),
 ('"7989C2AB3F345F4AB54D3CF1E0480D67"', 39),
 ('"6C9F67DF658DC5636F9E7752F203F70A"', 39)]
```

Рисунок 2 – Топ-10 популярных номеров такси

В завершении первой части лабораторной работы я узнал о способах хранения данных. Они включают в себя хранение в сериализованном виде, в

исходном, с репликацией и другие, а задаются вызовом метода persist() с соответствующим аргументом: MEMORY_ONLY, MEMORY_AND_DISK, MEMORY_AND_DISK_SER, DISK_ONLY, MEMORY_ONLY_2, MEMORY_AND_DISK_2, OFF_HEAP.

Вторая часть работы, как уже было сказано ранее, заключалась в проведении анализа данных о велопарковках. В данной части использовалось два набора данных: «stations.csv» – данные о станциях, «trips.csv» – данные о велосипедных поездках.

Первоначально была проведена предобработка данных. Была проведена соответствующая индексация, в качестве ключа при которой выступает идентификатор станции. А потом было проведено соединение двух таблиц по этому идентификатору. Таким образом удалось получить связи вида:

1. Поездка – данные о станции старта.
2. Поездка – данные о станции финиша.

Для более эффективной обработки и получения дополнительных возможностей были объявлены классы сущностей предметной области и преобразованы исходные строковые данные в объявленное представление. После этого были выполнены все поставленные задания.

Далее мне предстояло поработать с Spark Dataframe и Spark SQL. После инициализации Spark-контекста упомянутые ранее .csv файлы были считаны как Dataframe с автоматическим определением типов. Из последующих операций можно выделить удаление мусорных значений, а также объединение по идентификатору станции.

В этой части работы те же самые операции, которые делались через RDD, выполнялись с помощью Dataframe API и SQL-запросов. В конце также координаты станций переводились в индекс гексагональной сетки H3, затем были выбраны связанные поездки, и результат был визуализирован на карте через pandas и folium. Результат визуализации представлен на рисунке 3.

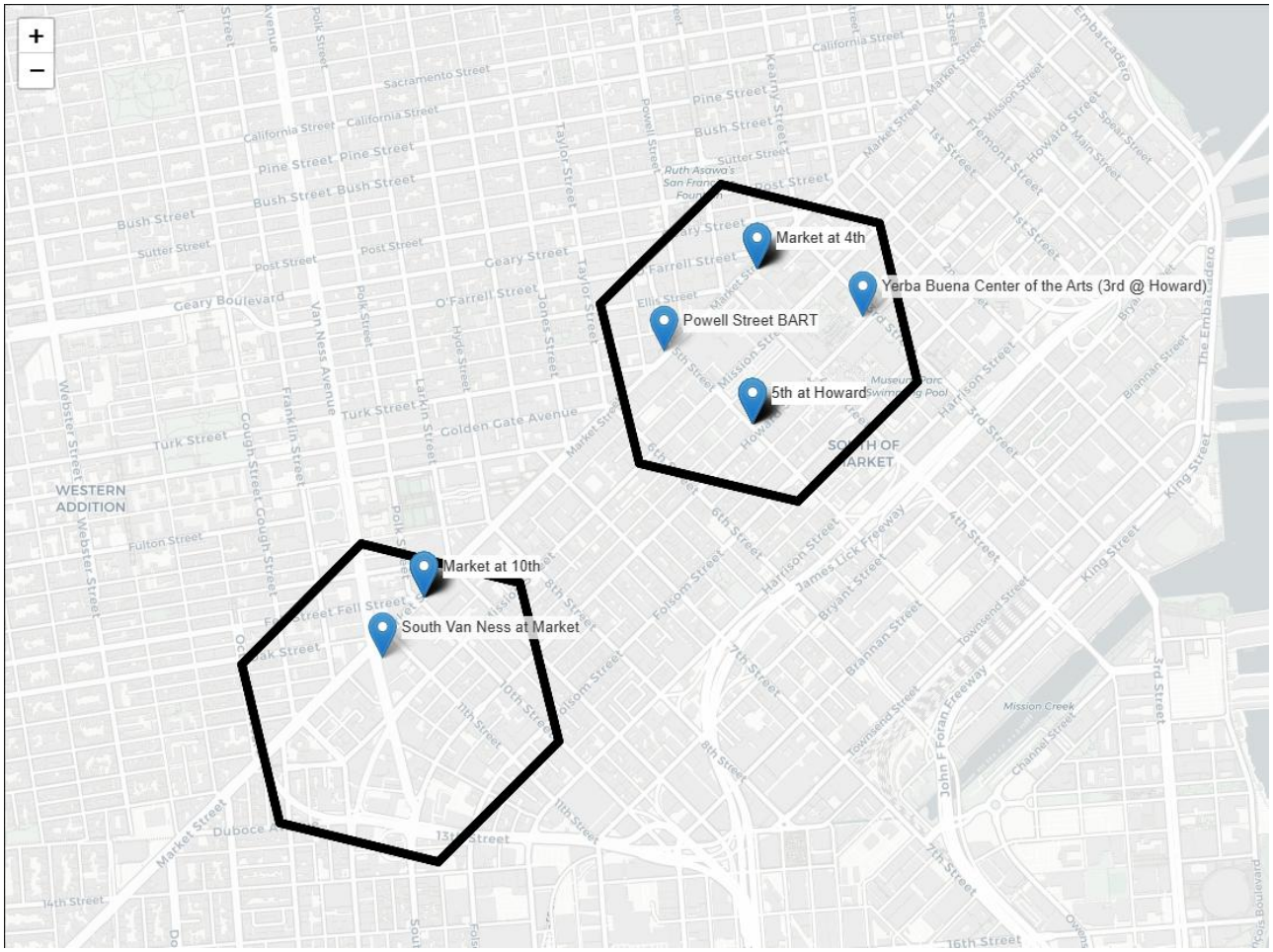

Рисунок 3 – Гео-визуализация данных в Spark

Также были выполнены задания по данному набору данных в интерактивном и не интерактивном режимах.

Для поиска велосипеда с максимальным пробегом были созданы пары: идентификатор велосипеда – длительность, далее было просуммировано время по каждому велосипеду, и найден максимум среди значений времени.

Максимальное расстояние между станциями было найдено следующим образом: построены все возможные пары станций, убраны дубликаты, рассчитано расстояния для каждой пары, и найден максимум среди всех значений расстояний.

Маршрут велосипеда с максимальным пробегом был построен путем сортировки по времени всех поездок велосипеда, найденного в первом задании.

Количество велосипедов было найдено с помощью ранее упомянутых distinct() и count() – убраны дубликаты и посчитано количество элементов.

Последнее задание – пользователи, потратившие на поездки более трех часов – было выполнено так: поездки сгруппированы по ZIP-коду, просуммирована длительность, и применен фильтр, оставляя тех пользователей, кто превысил заданное время.

Все выполненные задания можно найти в репозитории: https://github.com/artmkchmv/BigData/tree/main/Lab1

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы мною были изучены и практически применены механизмы обработки больших данных с использованием Apache Spark в среде WSL (Ubuntu 24.04) на Windows.

В первой части работы были освоены принципы работы с RDD: операции преобразования и действий, управление партициями, а также механизмы оптимизации через кэширование, broadcast-переменные и выбор стратегии хранения данных.

Во второй части работы были выполнены задачи анализа данных велопроката с использованием как RDD, так и более высокоуровневых инструментов Spark DataFrame и Spark SQL. Были реализованы операции объединения наборов данных, агрегации, фильтрации и анализа временных рядов.

Дополнительно были изучены геопространственные методы обработки данных с использованием H3-гексагональной сетки и выполнена визуализация результатов на карте с помощью folium.